

Smart Flood Monitoring & Early Warning System

Pin Connection Guide (Step-by-Step) + Arduino Code

Ei guide-e Arduino Uno-r sathe HC-SR04 ultrasonic sensor, I2C LCD (16x2), rain sensor, yellow/red LED, buzzer, ar SIM800L GSM module kivabe connect korte hobe seta step-by-step deya ache. Tinkercad ba real hardware, dutoi khetre same wiring apply hobe (GSM module real hardware chara Tinkercad-e simulate hoy na, tai Tinkercad-e GSM part optional).

1. Component List

Component	Quantity	Note
Arduino Uno	1	Main controller
HC-SR04 Ultrasonic Sensor	1	Water level measure korbe (distance)
16x2 I2C LCD Display	1	Address 0x27 (na cholle 0x3F try korben)
Rain Sensor Module	1	Digital output mode
Yellow LED	1	Safe/normal indicator
Red LED	1	Danger indicator
Buzzer (active)	1	Danger alarm
220 ohm Resistor	2	LED protection (yellow + red)
SIM800L GSM Module	1	SMS alert pathanor jonno
Breadboard + Jumper wires	-	Connection-er jonno
External 4V-4.4V power (GSM)	1	SIM800L Arduino 5V-e stable chole na

2. Full Pin Mapping (Arduino Uno)

Arduino Pin	Connect To	Component
5V	VCC	HC-SR04
GND	GND	HC-SR04
D9	Trig	HC-SR04
D10	Echo	HC-SR04
5V	VCC	I2C LCD
GND	GND	I2C LCD
A4 (SDA)	SDA	I2C LCD
A5 (SCL)	SCL	I2C LCD
5V	VCC	Rain Sensor Module
GND	GND	Rain Sensor Module
D8	D0 (Digital OUT)	Rain Sensor Module
D6	Resistor -> Anode (+)	Yellow LED

GND	Cathode (-)	Yellow LED
D4	Resistor -> Anode (+)	Red LED
GND	Cathode (-)	Red LED
D7	+ terminal	Buzzer
GND	- terminal	Buzzer
D2	TXD (GSM)	SIM800L GSM Module
D3	RXD (GSM)	SIM800L GSM Module
External 4V + common GND	VCC + GND	SIM800L GSM Module

Note: D2/D3 GSM_RX_PIN/GSM_TX_PIN naam confusing lagte pare - code-e `SoftwareSerial gsmSerial(GSM_RX_PIN, GSM_TX_PIN)` mane Arduino D2 = RX pin (GSM-er TX theke data receive kore), Arduino D3 = TX pin (GSM-er RX-e data pathay). Tai cross-connect: Arduino D2 <-- GSM TXD, Arduino D3 --> GSM RXD.

3. Step-by-Step Wiring Instructions

Step 1 - Power rail set up

Breadboard-e Arduino 5V ar GND, positive (+) ar negative (-) rail-e connect korun. Shob component-er common GND ei rail theke nite hobe, jate shobar GND same reference-e thake.

Step 2 - HC-SR04 Ultrasonic Sensor

VCC pin -> 5V rail. GND pin -> GND rail. Trig pin -> Arduino D9. Echo pin -> Arduino D10. Sensor-ta emonvabe bosaben jate eta sojasuji panir upor theke distance measure korte pare (water tank-er upore mount).

Step 3 - I2C LCD Display (16x2)

VCC -> 5V rail. GND -> GND rail. SDA -> Arduino A4. SCL -> Arduino A5. Default address code-e 0x27 deya ache; LCD blank thakle ba box dekhale I2C scanner code diye address check korben, onek shomoy address 0x3F hoy - shetai note-e likha ache je unresolved issue.

Step 4 - Rain Sensor Module

VCC -> 5V rail. GND -> GND rail. Digital OUT (D0) pin -> Arduino D8. Sensor board-er sensitivity potentiometer ghuriye threshold set korte paren. Code-e LOW hole 'rain detected' dhora hocche, tai digital mode-e rakhben.

Step 5 - Yellow LED (Safe indicator)

Arduino D6 -> 220 ohm resistor -> LED-er longer leg (anode, +). LED-er choto leg (cathode, -) -> GND rail.

Step 6 - Red LED (Danger indicator)

Arduino D4 -> 220 ohm resistor -> LED-er anode (+). Cathode (-) -> GND rail.

Step 7 - Buzzer

Buzzer-er positive terminal -> Arduino D7. Negative terminal -> GND rail. Active buzzer hole shudhu HIGH/tone dile shobdo hobe, passive hole `tone()` function lagbe (code-e `tone()` use kora hoyeche, tai active/passive dutoi mota-muti cholbe).

Step 8 - SIM800L GSM Module

GSM module-er RXD -> Arduino D3, TXD -> Arduino D2. IMPORTANT: SIM800L 5V-e directly power dile module heat hoye reset khete pare - external 4V-4.4V, 2A capable power supply use korun, ar Arduino-r GND-er sathe GSM power supply-r GND common kore dite hobe (common ground must). SIM card insert kore rakhben, network LED slow-blink na howa porjonto wait korben.

Step 9 - Final check before power on

Shob GND ekshathe common kina check korun. Kono short circuit ache kina dekhe nin. LCD, LED, buzzer polarity thik ache kina abar check korun. Tarpur USB/power connect kore code upload korun.

Threshold Note: Code-e DANGER_THRESHOLD_CM = 11 set kora - mane sensor theke water surface-er distance 11 cm-er kom hole DANGER state trigger hobe. Nijer tank/pond-er depth onujayi ei value change korte paren.

4. Full Arduino Code

Niche shompurno .ino code deya holo, jeta upore bola pin connection-er sathe match kore.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>

// ----- Pin definitions -----
const int trigPin = 9;
const int echoPin = 10;
const int yellowLED = 6;
const int redLED = 4;
const int buzzerPin = 7;
const int RAIN_SENSOR_PIN = 8;
const int GSM_RX_PIN = 2;
const int GSM_TX_PIN = 3;

// ----- Thresholds -----
const int DANGER_THRESHOLD_CM = 11;

const char *ALERT_PHONE_NUMBER = "+8801993914116";

// ----- Objects -----
LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial gsmSerial(GSM_RX_PIN, GSM_TX_PIN);

// ----- Variables -----
long duration;
int distance;

// ----- State -----
enum FloodLevel { SAFE, DANGER };
FloodLevel lastLevel = SAFE;
bool alertSmsSent = false;

void setup() {
  Serial.begin(9600);
  gsmSerial.begin(9600);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(RAIN_SENSOR_PIN, INPUT);

  lcd.init();
  lcd.clear();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Flood Monitor");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);
  lcd.clear();

  initGSM();
}

void loop() {
  distance = readUltrasonicDistance();
  bool isRaining = (digitalRead(RAIN_SENSOR_PIN) == LOW); // LOW = Rain detected

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  FloodLevel level = classifyLevel(distance);

  updateLCD(distance, isRaining, level);
  updateOutputs(level);
  handleAlertSms(level);

  lastLevel = level;
}
```

```

    delay(1000);
}

// ----- Ultrasonic -----
long readUltrasonicDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0) return 999;
    return duration * 0.034 / 2;
}

// ----- Logic -----
FloodLevel classifyLevel(int distanceCm) {
    if (distanceCm < DANGER_THRESHOLD_CM) {
        return DANGER;
    }
    return SAFE;
}

// ----- Outputs -----
void updateOutputs(FloodLevel level) {
    digitalWrite(yellowLED, level == SAFE);
    digitalWrite(redLED, level == DANGER);

    if (level == DANGER) {
        tone(buzzerPin, 2000);
    } else {
        noTone(buzzerPin);
    }
}

void printPadded16(String text) {
    while (text.length() < 16) text += " ";
    if (text.length() > 16) text = text.substring(0, 16);
    lcd.print(text);
}

void updateLCD(int distanceCm, bool isRaining, FloodLevel level) {
    lcd.setCursor(0, 0);
    printPadded16(isRaining ? "Rain: Yes" : "Rain: No");

    lcd.setCursor(0, 1);
    String levelText = (level == DANGER) ? "DANGER!" : "Safe";

    printPadded16("D:" + String(distanceCm) + "cm " + levelText);
}

// ----- GSM / SMS -----
void initGSM() {
    gsmSerial.println("AT");
    delay(500);
    gsmSerial.println("AT+CMGF=1");
    delay(500);
}

void handleAlertSms(FloodLevel level) {
    if (level == DANGER && lastLevel != DANGER && !alertSmsSent) {
        sendSMS("FLOOD ALERT! Water level is critical.");
        alertSmsSent = true;
    }

    if (level == SAFE) {
        alertSmsSent = false;
    }
}

void sendSMS(const char *message) {
    gsmSerial.println("AT+CMGF=1");
    delay(300);
    gsmSerial.print("AT+CMGS=\"");
    gsmSerial.print(ALERT_PHONE_NUMBER);
    gsmSerial.println("\");
}

```

```
    delay(300);  
    gsmSerial.print(message);  
    delay(300);  
    gsmSerial.write(26);  
    delay(3000);  
}
```